TDM (TEST DATA MANAGEMENT) UPGRADE PROCEDURE TO V10.X

This document describes the following:

How to upgrade TDM to the current version **-** 10.X

How to reimplement the modified product's features.

Notes:

This document does not cover the Fabric server topology changes, such as the addition of nodes, data centers, changes to replication factors or consistency levels.

The TDM upgrade procedure should be performed on testing environments prior to applying it on your production deployment.

Perform a sanity test upon completion of the upgrade procedure, such as running a few TDM tasks and conducting other checks per the sanity procedure defined in your project.

SOFTWARE UPGRADE PROCEDURE

1 TDM 10.X Installation — Prerequisites

Upgrade to Fabric version 8.4.x or higher.

2 Related Documents

FABRIC UPGRADE PROCEDURE TO V8.3

Note that Step 1 of the Fabric Upgrade Procedure document is irrelevant for a TDM project as the latter does not contain the iidFinder process.

For more information about the TDM 10.X installation, please read the TDM Installation article in the TDM Configuration.

3 TDM Upgrade Steps

i. Fabric Upgrade

ii. Upgrade TDM Code (TDM Library)

- Development Environment — Web Studio
- Development Environment – Desktop (.NET) Studio
- Non-Development Environment

iii. TDM Project and DB Upgrade

- General Prerequisites
- Development Environment
- Non-Development Environment

iv. Catalog Setting — Rebuilding Artifacts

4 TDM App Setup

4.1 TDM 10 Post-Upgrade Configuration

The following steps are optional. TDM 10 can operate without these steps; however, completing them is recommended to optimize the configuration and user experience.

4.1.1 Organizing Existing Tasks into Task Groups

- TDM 10 organizes tasks into logical groups based on domain, team, application, or use case, making task navigation simpler and more intuitive.
- **During the upgrade, all existing tasks are automatically assigned to the *General* task group** to ensure a seamless transition. While no additional action is required, we recommend creating additional task groups and moving existing tasks into them.
- Organizing tasks into smaller, logical groups provides the following benefits:
  - Simplifies task discovery and navigation in the TDM App.
  - Improves scalability by reducing the number of tasks displayed within a single group.
  - Enhances performance, as the TDM App retrieves only the tasks belonging to the selected task group.
- Plan the task group structure according to your organization's domains, teams, applications, or use cases, and distribute existing tasks accordingly.

4.1.2 Reviewing Task Access Permissions

- After upgrading to TDM 10, review the access permissions for your existing tasks.
- By default, an upgraded task can be executed by all TDM users who are assigned to TDM environments with permission sets that allow task execution.
- Edit the task's **Access Control** settings to change the task to be accessed and limit its execution.

4.1.3 Configuring Runtime-Editable Task Attributes

- After upgrading to TDM 10, review your existing tasks and configure which task attributes are locked and which can be edited by the task runner at execution time.

v. Additional Steps for Upgrades from TDM 9.0 or Earlier

vi. Optional: Parameter Coupling Mode - Change the Parameters mode to Parameters Coupling

5 Fabric Upgrade

Upgrade the Fabric version to 8.4.x or higher.

If your current Fabric version is older than 8.0, perform the following:

Before upgrading the project, edit the config.ini file and uncomment the PACKAGE_NAMES_CLASS_LOADING_FILTER parameter. Set it as empty to disable the isolation feature between Fabric's dependencies and the project's dependencies.

After upgrading the project, the PACKAGE_NAMES_CLASS_LOADING_FILTER parameter needs to be commented.

For more information, read FABRIC UPGRADE PROCEDURE TO V8.0.

6 Upgrade TDM Code (TDM Library)

Development Environment — Web Studio

Development Environment – Desktop (.NET) Studio

Non-Development Environment

5.1 Development Environment — Web Studio

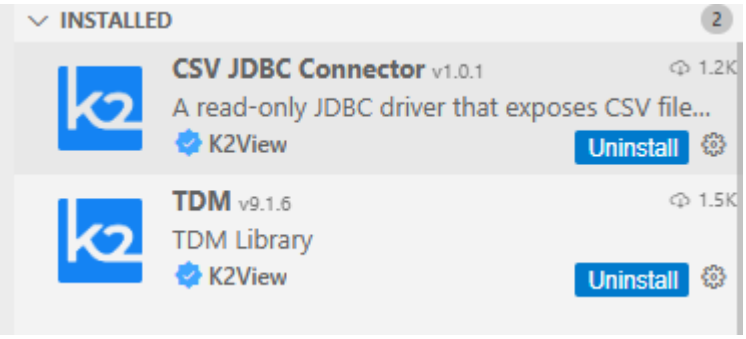
TDM Extension Installation

VSIX File Installation

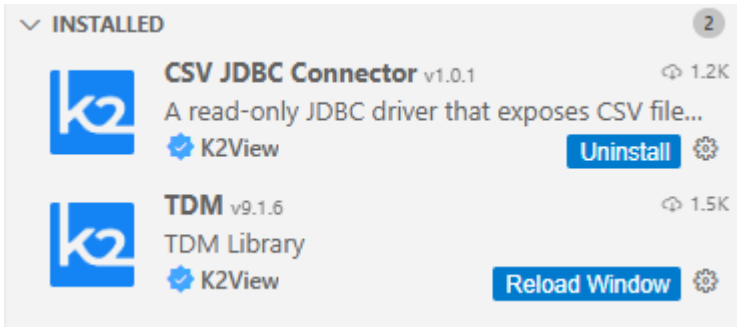
5.1.1 TDM Extension Installation

5.1.1.1 The Source TDM Version is Installed via Extension

- Open the TDM project in Fabric Studio. The Fabric Studio will upgrade the code to the latest Fabric version.
- Click the Extensions icon ( ), select the TDM under the INSTALLED list and uninstall the old TDM version:



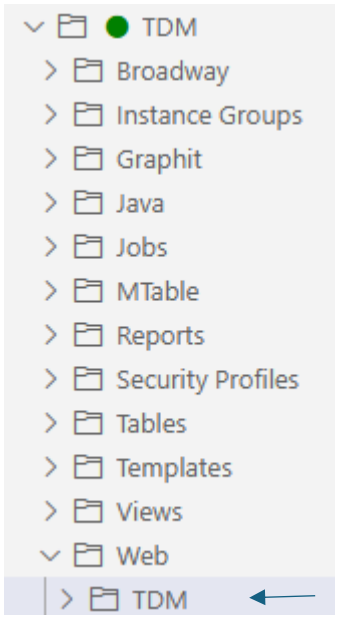
- Click the Reload Window button next to the uninstalled TDM version:



- Click the Extensions icon — — and select TDM to install the TDM 10.X library. **Override the existing objects.**
- Redeploy TDM and the TDM_TableLevel LUs.

5.1.1.2 The Source TDM Version is not Installed via Extension

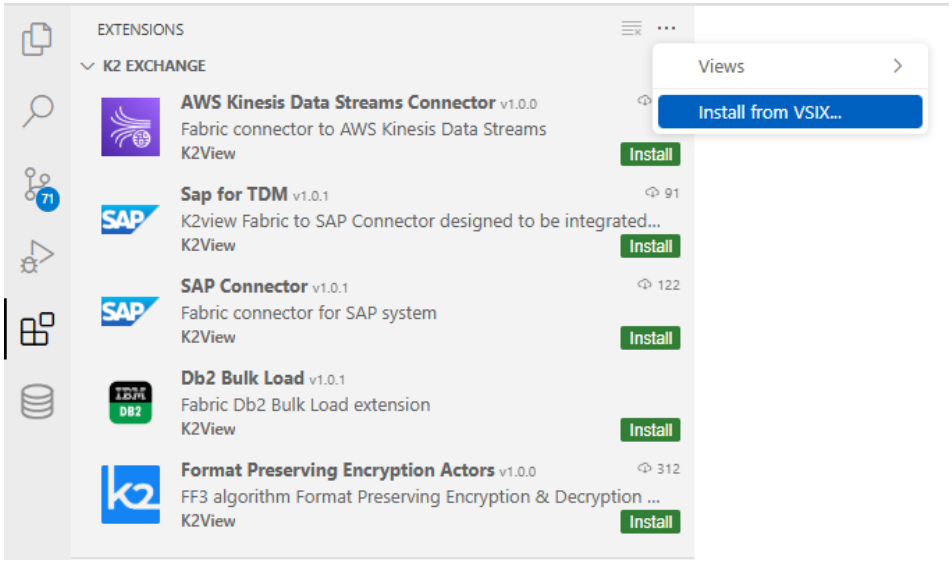
- Open the TDM project in Fabric Studio. The Fabric Studio will upgrade the code to the latest Fabric version.
- Delete the TDM subfolder located under the TDM/Web folder in order to delete the current version of the TDM App from the TDM LU **before** clicking the Extensions icon:



- Note that you **must not delete the entire TDM LU** from the project if you wish to **keep the history of previous task executions in your development environment**. Deleting the TDM LU from the project would delete it from Fabric and remove the execution history stored in the local Fabric.
- Click the Extensions icon — — and select TDM to install the TDM 10.X library. **Override the existing objects.**
- Redeploy TDM and the TDM_TableLevel LUs.

5.1.2 VSIX File Installation

- If access to the Extensions is not available, install the VSIX file. Download the VSIX file from the download page and upload it to the TDM project: Right-click the project-resources from the Project tree > Upload Files… > select the TDM VSIX file.
- Click the Extension icon — — and then click the … icon (top-right of the pane) and select the 'Install from VSIX…' option:



-
-
- A pop-up window opens: Select the uploaded TDM VSIX file and click the Install from VSIX button. Override the existing objects in your project.
- Redeploy TDM and the TDM_TableLevel LUs.
-

5.2 Development Environment – Desktop (.NET) Studio

Step 1 – Back up the Project's Populated TDM Objects

- Back up the following objects in your project:

CustomLogicFlows.actor

TDMFilterOutTargetTables.actor

TDMSeqList.actor

TDMSeqSrc2TrgMapping.actor

TDMTargetTablesNames.actor

TableLevelInterfaces.csv

Step 2 – Open the TDM Project in Fabric Studio

- Open the TDM project in Fabric Studio. The Fabric Studio will upgrade the code to the latest Fabric version.
- If your current TDM version is older than 9.0, follow these steps:

- Copy the following .jar files into the \K2View Fabric Studio\Projects\<project name>\lib folder:

json-20231013

handlebars-4.3.0

cron-utils-9.2.1

commons-lang3-3.11

- Note that the commons-lang3-3.11.jar file is needed only for the upgrade flows. Remove it from the lib folder following completion of the TDM upgrade process as the TDM 9.X code no longer uses the StringUtils class.

-
- Manually delete the following:

TDM LU

TDM_LIBRARY LU

If your current TDM version is 9.0 or older, delete the TDM_Reference LU. Else, delete the TDM_TableLevel LU.

Step 3 – Import the TDM Library into the Project

-
- Import the TDM LUs export file into your project using the 'Import All' option for importing the following LUs:

TDM

TDM_LIBRARY LU

TDM_TableLevel LU

- Import the **Web Services** into your project using the Custom Import… option.
- Import the following **Shared Objects** into the Fabric project using the Custom Import… option:

Templates

Broadway

Java

- If your current TDM version is 9.0, use the Custom Import… option to import the TableLevelDefinitions.csv **MTable** into the project under the **References** LU.
- Note that the TableLevelInterfaces.csv will be updated by the TDM deploy flow.
- Optional — AI Interfaces:
  - If you would like to add the AI-based generation configuration to the TDM project, perform the following steps:
    - Import and edit the AI interfaces to the TDM project in the Studio.
    - Add the AI environment to the Studio.

- For more information, see AI implementation article.

Step 4 – Additional Steps – Needed if the Current TDM Version is Older than 9.X

- A new Global has been added to TDM V8.1 for masking and sequence handling — SEQ_CACHE_INTERFACE. This Global is populated with the DB interface of the k2masking DB (PostgreSQL or Cassandra) and must be aligned with Fabric's system DB. TDM V9 sets the **POSTGRESQL_ADMIN** as the **default** value in this Global:
  - If you use **Cassandra** as Fabric's system DB, you must edit the SEQ_CACHE_INTERFACE Global and update its value to **DB_CASSANDRA**.
  - If you wish to use the **PostgreSQL** DB as Fabric's system DB, perform the following steps:
    - Open Fabric's **config.ini** file and edit the **[system_db]** section's attributes, including the SYSTEM_DB_DATABASE attribute, to be aligned with the **POSTGRESQL_ADMIN** DB interface**.**
    - Restart Fabric.

- Removing duplicate TDM objects from the project's Shared Objects: The import process creates duplicate objects in the project since TDM V9 stores TDM objects in subfolders within the **Shared Objects**' Broadway and Java folders. The duplicated objects will be removed in the next step, i.e., running the UpgradeTDMProjectToTDM9 flow.

5.3 Non-Development Environment

-
- Prerequisite: The upgraded TDM project must be committed to GitHub.
- Pull the updated TDM project from GitHub

7 TDM Project and DB Upgrade

General Prerequisites

Development Environment

Non-Development Environment

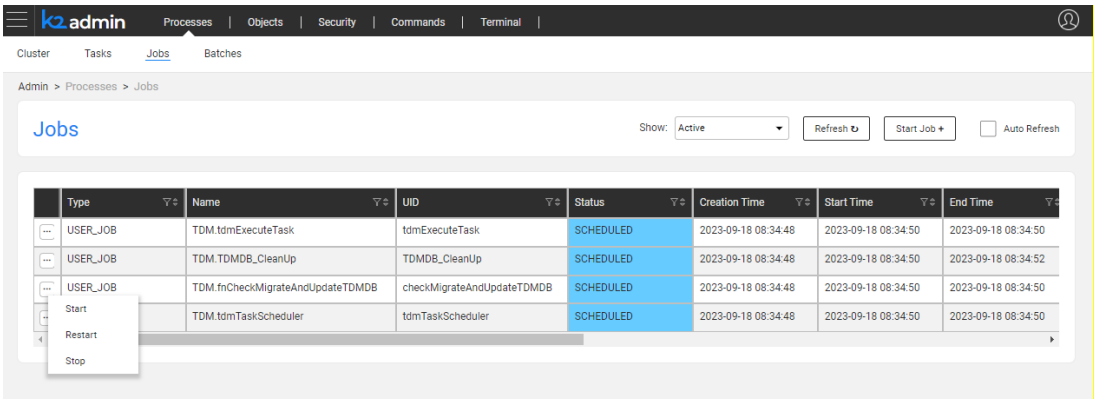
7.1 General Prerequisites

If your current version is 9.3.1, add the TDMDB_SCHEMA Global to the TDM Shared Global files.

Verify that the TDMDB_SCHEMA Global is correctly defined with the appropriate value for the TDM DB schema, and the TDM interface is properly set before the upgrade.

Open and save the Environments file in Fabric Studio and redeploy it to Fabric.

Stop all TDM jobs on Fabric, if any are running, to prevent the TDM DB tables from being locked by parallel execution of the upgrade flow and TDM jobs. Use the Web Admin to stop the TDM jobs:



Back up the TDM DB before the upgrade.

7.2 Development Environment - Project Implementation Changes

7.2.1 New in TDM 10

- The following capabilities were introduced in TDM 10 and may require post-upgrade configuration:
  - Updated task creation and execution APIs.
  - Task Groups for organizing tasks.
  - Runtime-editable task attributes.
  - Enhanced task access control.
  - Pre- and post-processes for table-level tasks.

7.2.2 TDM 10 - TDM APIs

- Task creation, edit, and execution APIs have been changed in the TDM 10:
- Task creation and edit APIs now get the list of editable and locked task attributes to indicate which task attributes can be edited by the task runner at execution.
- The input of the overridden execution parameters has been changed in the Task execution API (startTask API).
- If you call directly to these APIs in your code, the following changes are required:
- startTask API –

- If your code populates the overridden parameters and wishes to use the previous TDM versions manner “as is”: invoke V1 of the *startTask* API. This version builds the JSON for the *overrideParameters* input parameter.
- If your code does not override the execution parameters (e.g. source and target environments) and executes the task “as is”, then no code change is needed.
- The list of overridable attributes has been extended by TDM 10. If you wish to use the TDM 10 functionality, you need to update your code to be aligned with the TDM 10 structure.
- For more information see **TDM_API_Migration_Guide_v9.5_to_v10** document.

7.2.3 TDM 10 - Enabling pre and post processes on Table-Level tasks

- TDM 10 supports adding pre and post processes to table-level tasks.
- Populate the TableLevelPostAndPreExecutionProcess MTable and add the required pre and post processes and execution order to enable adding them to table-level tasks.

7.2.4 Additional Steps for Upgrades from TDM 9.4 or Earlier

- Update the load flows — populate the ‘table’ input argument of the ’Load Stats To TDM Table’ Actor. This fix is needed in order to display tables with zero records in the ‘Statistics Report’ tab of the TDM execution report (ticket #43620).
- Adding a description to the business parameters — a new field has been added to the LUParams and LuParamsMapping MTables — description. The description can be added and displayed on each business parameter in the Task window when selecting an entity subset based on business parameters.
- Adding Catalog-based masking to the LU population — if LU population flows do not include the CatalogMaskingMapper Actor, it should be added to the population flows to enable Catalog masking. Get the value from the SEQ_CACHE_INTERFACE Global and send it to the interface parameter of the CatalogMaskingMapper Actor.
- If you have the TDMOrchestrator flow under an LU, perform one of the following actions:
- Delete the TDMOrchestrator from your LU and use the one in the Shared Object instead.
- Update the TDMOrchestrator flow — use the DeleteFromTarget and LoadToTarget flows instead of the DeleteAllTables and LoadAllTables flows. This update is required since the upgrade flow renames the DeleteAllTables and LoadAllTables flows to DeleteFromTarget and LoadToTarget.

- **TableLevelDefinitionsInterfacesTypesDefaults** has been added in TDM 9.5 to the **TDM_TableLevel LU**. This MTable defines the default handling of table partitions for **PostgreSQL** and **Oracle** databases. If required, you can copy these records into the **TableLevelDefinitions** MTable.

7.2.4.1 TDM Cleanup Configuration

- **TDMCleanUp** MTable has been moved from TDM **LU** to the **Reference LU in TDM 9.5**. A new field, **OVERRIDE_CLEANUP_RETENTION_PERIOD**, has been added to this MTable. This field overrides the default cleanup retention period for specific tables.
- By default, it is set to 6 months for the TASK_EXECUTION_SUMMARY and TASK_REF_EXE_STATS tables.
- A longer retention period is defined for these tables because their data is required for **TDM Usage Reports**.
- You can configure a different cleanup period for these TDM tables if needed.

7.2.5 Additional Steps for Upgrades from TDM 8.x

- Deploy all LUs of the project to Fabric.
- Use the soft deploy option to deploy the TDM LU to Fabric without starting TDM jobs or executing the TDM deploy flow.
- Open and run the **UpgradeTDMProjectToTDM9** flow in order to update the TDM project with the updated TDM library, convert the legacy TDM translations to MTables, and upgrade the TDM DB.
- Note:
    - The upgrade process retrieves the **current TDM version** from the **tdm_general_parameters** TDM DB table.
- Deploy (regular deployment) the TDM LU to Fabric.
- Continue with other upgrade activities as described in Development Environment - Project Implementation Changes sections.

7.2.5.1 Optional — Update the Tasks with the Creator's Fabric Role

- This step is needed when the users are managed by an external IDP (e.g., SAML). This step should be implemented if you wish the TDM App to enable executing tasks created by all users, including users that belong to the task creator's group (Fabric role).

- From TDM V9.0 onwards, the user's Fabric role is concatenated to the username in the Tasks TDM DB table. This is required for the purpose of identifying the task creator's Fabric role and deciding whether a tester user can execute a task created by another user, when all users are managed and kept by the organization's IDP.
- Populate the **UserRolesUpgrade** MTable with the list of the TDM users and their Fabric roles before running the **TDMDBUpgradeScripts** flow. This flow will concatenate the user's Fabric role to the **task_created_by** and the **task_last_updated_by** fields of the Tasks TDM DB table.
- Redeploy the **References** LU.
-

7.2.6 Additional Steps for Upgrades from TDM 9.1 or Earlier

- Open the **TDMFilterOutTargetTables** Actor and add the Boolean column — **generator_filterout** — if it is missing. Set it to **true** for all TDM product tables and for the _TAR table.
- Open **CustomLogicFlows** Actor and add the Boolean column — **DIRECT_FLOW** — if it is missing. Leave this field clear for the existing records.
- For more information, see Custom Logic implementation article.
- Continue with other upgrade activities as described in Development Environment - Project Implementation Changes sections.
-

7.2.7 TDM DB Upgrade

- The TDM DB upgrade is initiated by the TDM LU deployment. Deploy (regular deployment) the TDM LU to Fabric. The TDM deployment upgrades the TDM DB.

7.2.7.1 TDM 10 - Optional – Updating Existing Tasks to Support Runtime Attribute Override

- TDM 10 introduces a unified mechanism for overriding task attributes at execution time, whether the task is executed from the TDM App or by invoking the ***startTask*** API directly.
- To preserve runtime the behavior of tasks created in previous TDM versions, run the ***UpdateTaskOverrideFields*** flow. This flow updates existing tasks by enabling runtime editing for the task attributes that were previously supported as execution parameters when calling the startTask API directly.
- After running the flow:
    - The relevant task attributes become editable during task execution in the TDM App.

  - The same attributes can also be overridden when invoking the startTask API directly.
  - No task logic is modified; only the runtime editability of the supported task attributes is updated.
  - For the list of task attributes that can be overridden at execution time, see Overriding Additional Task Execution Parameters.
- Note: This flow is intended for existing tasks that were created before upgrading to TDM 10. New tasks created in TDM 10 already support the new runtime override mechanism and do not require this update.
- Click here for more information about the task attributes handled by this flow.

7.3 TDM DB - Change Fabric Storage to a Type that does not Support TTL

- TDM enables creating tasks with a retention period (TTL) on the task's entities as a way of saving these entities in Fabric only for a limited period. However, if the Fabric storage does not support TTL for the LUIs (such as PG DB), TDM needs to limit the TDM task's retention period options to either 'Do not Delete' or 'Do not Retain'.
- Run the following steps to limit the TDM retention period:
  I. Update the tdm_general_parameters TDM DB to limit the TDM task's retention period options to either 'Do not Delete' or 'Do not Retain'.

  View the Update statements in https://support.k2view.com/Academy/articles/TDM/tdm_configuration/02_tdmdb_general_parameters.html

  II. Open the TDM App, then open the TDM tasks and update them with a retention period other than 'Do not Delete' or 'Do not Retain'.

7.4 Non-Development Environment

- Prerequisite: The upgraded TDM project must be committed to GitHub.
- Pull the updated TDM project from GitHub and deploy the updated TDM LU. The TDM deployment upgrades the TDM DB.
- Redeploy the entire project, including the project's LUs and Web Services.
- TDM DB - Change Fabric Storage to a Type that does not Support TTL. For more information, see TDM DB - Change Fabric Storage to a Type that does not Support TTL section.

7.4.1.1 TDM 10 - Optional – Updating Existing Tasks to Support Runtime Attribute Override

- TDM 10 introduces a unified mechanism for overriding task attributes at execution time, whether the task is executed from the TDM App or by invoking the ***startTask*** API directly.
- To preserve the runtime behavior of tasks created in previous TDM versions, run the ***UpdateTaskOverrideFields*** flow. This flow updates existing tasks by enabling runtime editing for the task attributes that were previously supported as execution parameters when calling the startTask API directly.
- For more information, see TDM 10 - Optional – Updating Existing Tasks to Support Runtime Attribute Override section.
-

8 Catalog Setting — Rebuilding Artifacts

Starting with V8.3, Fabric creates a separate catalog_field_info.csv file for each interface. Delete the old catalog_field_info.csv from the project and rebuild the artifacts in the Catalog for the required version to add the new Catalog files to the TDM project.

-

9 TDM App Setup

9.1 TDM 10 Post-Upgrade Configuration

The following steps are optional. TDM 10 can operate without these steps; however, completing them is recommended to optimize the configuration and user experience.

9.1.1 Organizing Existing Tasks into Task Groups

- TDM 10 organizes tasks into logical groups based on domain, team, application, or use case, making task navigation simpler and more intuitive.
- **During the upgrade, all existing tasks are automatically assigned to the *General* task group** to ensure a seamless transition. While no additional action is required, we recommend creating additional task groups and moving existing tasks into them.
- Organizing tasks into smaller, logical groups provides the following benefits:
    - Simplifies task discovery and navigation in the TDM App.
    - Improves scalability by reducing the number of tasks displayed within a single group.
    - Enhances performance, as the TDM App retrieves only the tasks belonging to the selected task group.
- Plan the task group structure according to your organization's domains, teams, applications, or use cases, and distribute existing tasks accordingly.

9.1.2 Reviewing Task Access Permissions

- After upgrading to TDM 10, review the access permissions for your existing tasks.
- By default, an upgraded task can be executed by all TDM users who are assigned to TDM environments with permission sets that allow task execution.
- Edit the task's **Access Control** settings to change the task to be accessed and limit its execution.

9.1.3 Configuring Runtime-Editable Task Attributes

- After upgrading to TDM 10, review your existing tasks and configure which task attributes are locked and which can be edited by the task runner at execution time.

9.2 Additional Steps for Upgrades from TDM 9.0 or Earlier

- TDM V9 does not support selecting the Delete option together with either the Replace IDs for the copied entities option or the Generate clones for an entity option.
- If you have tasks with Delete and Load task actions, whose Replace Sequence is selected or their selection method is entity clone, open them in the TDM App before running the upgrade flow, and update them as follows:
  - Delete + Load + Replace Sequence task — clear either the Delete or the Replace Sequence option.
  - Delete + Load + Entity clone — clear the Delete option.
- TDM V9 changes the way tables are stored in Fabric. The tables are now stored in a new LU — **TDM_TableLevel**. The previous LU — **TDM_Reference** — is no longer in use.

9.3 Additional Steps for Upgrades from TDM 8.0 or Earlier

9.3.1 TDM App – Resaving Tasks with Parameters Selection Method

- TDM V8.1 changed the way tasks with Parameters selection method are saved in the TDM DB. It is therefore required to open and resave the TDM tasks with the Parameters selection method after upgrading TDM and before executing the TDM tasks.

9.3.2 Rerun an Extract Task to Repopulate the Tables of LU Parameters

-
- The upgrade script **updates** the **<LU name>_params table, and it is based on the task_execution_entities**. By default, the task_execution_entities table contains executions of only the last 7 days (=0.25 month). Consequently, **the <LU name>_params table will also contain the entities of only the last 7 days of executions** (if the related task_execution_entities record is not found, the upgrade job would delete the related <LU name>_params record as well).

- If the <LU name>_param table must contain execution history longer than the last 7 days, rerun an Extract task on a large population following completion of the TDM upgrade process as a way to repopulate the missing <LU name>_params records.

10 Parameter Coupling Mode - Change the Parameters mode to Parameters Coupling

From TDM V9.1 onwards, when the selection of an entity subset for a TDM task is based on business parameters, it can be based on the newly added mode — *Parameters Coupling*.

Click here for more information about the Parameters Coupling mode.

The following steps should be taken if you would like to set the parameter's mode to Parameters Coupling:

10.1.1 Set, Create and Alter Schema and Table Permissions for the TDM User

The Parameters Coupling mode uses the MDB export Fabric command to export the parameter information of each LU into a dedicated schema in the TDM DB. A separate schema is created in the TDM DB for each LU.

Verify that the TDM DB user, which is set in the TDM interface, has permissions to create and edit schemas and tables.

10.1.2 Run the UpgradeToParamsCouplingMode Flow

The UpgradeToParamsCouplingMode flow —

Updates the TDM_GENERAL_PARAMETERS TDM DB table — sets the PARAM_COUPLING parameter to 'true'.

Creates a backup table for the tdm_params_distinct_values TDM DB table.

Truncates the tdm_params_distinct_values in the TDM DB table.

Renames the <lu>_params tables in the TDM DB — adds a _bck suffix to these tables since they are no longer needed for the Parameters Coupling mode.

Converts the LuParams.csv to the LuParamsMapping.csv, where applicable.

Adds the TDM_BE_IIDS LU table to the LUs.

Updates the FABRIC_TDM_ROOT table in the LU — adds a PK to this table in order to support the MDB export of the LU into the TDM DB.

10.1.3 Update the LU Implementation

Verify that the linked fields are defined as either PKs or unique indexes in the parent LU table for supporting the MDB export of these tables. All parent LU table's PKs/unique index fields must be linked to the child LU table. This is required when creating the FK relation in the PG DB for the exported LU tables.

Verify that the linked fields in the LU tables have identical data types. This is required as it supports the MDB export of the LU schema into the TDM DB.

Add a table to the LU for calculated parameters. For example, the total open debt amount is based on the accumulation of all open invoices. Each parameter in the Parameters Coupling mode must be mapped to a field in an LU table. Unlike in the regular mode, in the Parameters Coupling mode you cannot define an SQL query to get a parameter from the LuParamsMapping Mtable.

This new table with the calculated parameters should be added to the **TDMFilterOutTargetTables.actor** in a way that it would be excluded from creating the load, delete, and data generation flows for it.

Verify that all the LU tables in the LuParamsMapping are linked to parent tables. This is required when creating FKs on the exported tables during their export to TDM DB.

Update the LuParamsMapping.csv MTable — add the parameters that are based on the newly created business tables.

Redeploy the implementation, including **References** LU.

10.1.4 Rerun the Extract Tasks

Re-extract an entity subset for each Business Entity (BE) as a means to:

Create LU schemas in the TDM DB and export the entities' data into these schemas.

Repopulate the tdm_params_distinct_values in the TDM DB table.